

Question de cours:

Le langage de programmation C est un langage ?

A .Humain B. Machine C.Haut niveau D. Interprété

Le langage C est un langage de programmation **haut niveau**, car il permet de programmer en utilisant des instructions proches du langage humain, tout en offrant une bonne proximité avec le matériel grâce à son efficacité.

Lequel des identificateurs suivants n'est pas acceptable en langage C ?

A ._8b B._b8 C._B8 D. Toutes les réponses

aucune réponse n'est inacceptable.

Question de cours:

```
#include<stdio.h>
int main(){
int x=2, z=2 ;
z=z/++x ;
printf("%f ",z) ; }
```

A .0.000000 B.1 C. Une erreur D. 0.666666

```
main.c:6:14: warning: format '%f' expects argument of type 'double', but argument 2 has type 'int' [-Wformat=]
   6 |         printf("%f", z);
     |               ~^  ~
     |               |  |
     |               |  int
     |               double
     |               %d
0.000000
```

Question de cours:

```
1 void main () {  
2     int a=1, b=0, c = 1;  
3     if (!(a>c) || (a+b)&&(a-c) || (a || b))  
4         printf("faux");  
5     else  
6         printf("vrai");  
7 }
```

A .faux B.vrai C. vrai faux D. faux vrai

Question de cours:

```
#include<stdio.h>
int main(){
    int x=5, y=2 ;
    y=++x; y+x;
    printf("%d", y) ;
}
```

A. 7 **B. 6** C. 8 D. 5

- L'instruction `y + x;` n'a aucun effet sur le programme car le résultat de l'addition n'est ni stocké ni utilisé.
- L'incrément préfixe `++x` modifie la valeur de `x` avant de l'affecter à `y`.
- La sortie est uniquement déterminée par l'affectation de `++x` à `y`.

Exercice 1 :

```
#include<stdio.h>

int g(int );

void f(void );

int n=10,q=2;

int main()
{   int n=0,p=5;
    n=g(p);
    printf("A : dans main, n = %d, p = %d, q = %d\n ", n ,p, q);
    f();
}

int g(int p)
{   int q;
    q=2*p+n;
    printf(" B : dans g, n = %d, p = %d, q = %d\n",n,p,q);
    return q;
}

void f( void)
{   int p=q*n;
    printf (" C : dans f, n = %d, p = %d, q=%d\n",n,p,q);
}
```

B : dans g, n = 10, p = 5, q = 20
A : dans main, n = 20, p = 5, q = 2
C : dans f, n = 10, p = 20, q=2

Exercice 2 :

La suite de conjecture de Syracuse est une fonction mathématique qui repose sur un principe simple. On part d'un nombre entier n strictement positif :

s'il est **pair**, on le divise par 2 ;

s'il est **impair**, on le multiplie par 3 et on ajoute 1.

En effectuant cette opération plusieurs fois, on obtient une série d'entiers qui se termine toujours à la valeur 1.

Par exemple, la valeur $n=17$ donne la suite suivante : 52 26 13 40 20 10 5 16 8 4 2 1.

Ecrire une fonction en C **syracuse** qui prend en paramètre un nombre n , et qui affiche les valeurs successives de la suite relative à ce nombre, en s'arrêtant à la valeur 1.

Ecrire le programme principal qui teste la fonction syracuse.

Solution

```
#include <stdio.h>

// Fonction Syracuse
void syracuse(int n) {
    printf("La suite de Syracuse pour n = %d :\n", n);

    // Tant que n n'est pas égal à 1
    while (n != 1) {
        printf("%d ", n);

        // Si n est pair
        if (n % 2 == 0) {
            n = n / 2;
        } else {
            // Si n est impair
            n = 3 * n + 1;
        }
    }
    // Afficher la dernière valeur (1)
    printf("%d\n", n);
}
```


Solution

```
int main() {  
    int nombre;  
  
    printf("Entrez un nombre entier strictement positif : ");  
    scanf("%d", &nombre);  
  
    // Vérification si le nombre est valide  
    if (nombre > 0) {  
        syracuse(nombre); // Appel de la fonction Syracuse  
    } else {  
        printf("Veuillez entrer un nombre strictement positif.\n");  
    }  
  
    return 0;  
}
```

Exercice 3 :

Soient deux tableaux d'entiers A (maximum 50) et B (max 100). Ecrire un programme C qui permet de vérifier que B existe dans A. Si B existe dans A, alors supprimer les éléments de B dans A.

NB: Utiliser le formalisme pointeur.

Exemple :

A:

1	11	15	8	0	9	13	23	100
---	----	----	---	---	---	----	----	-----

B :

15	8	0	9	13
----	---	---	---	----

- Si B existe dans A alors le tableau A devient :

1	11	13	23	100
---	----	----	----	-----

Solution

```
#include <stdio.h>
int main() {
    int A[50], n ; // Tableau A
    int B[100], m , i; // Tableau B

    int *ptrA = A; // Pointeur sur A
    int *ptrB = B; // Pointeur sur B

    // saisie de n
    printf("Entrez la taille du tableau: ");
    scanf("%d%d", &n, &m);

    // remplissage du tableau A
    for (i=0; i<n; i++)
    {
        printf("Entrez tableau A: ");
        scanf("%d", &A[i]);
    }

    // remplissage du tableau B
    for (i=0; i<m; i++) {
        printf("Entrez tableau B: ");
        scanf("%d", &B[i]);
    }
}
```

```

// Parcourir A avec pointeurs
for (int i = 0; i < n; i++) {
    int trouve = 0; // Indicateur pour vérifier si un élément de A existe dans B

    // Parcourir B pour vérifier si *(ptrA + i) existe dans B
    for (int j = 0; j < m; j++) {
        if (*(ptrA + i) == *(ptrB + j)) {
            trouve = 1; // Élément trouvé
            break;
        }
    }
    // Si l'élément est trouvé dans B, supprimer cet élément de A
    if (trouve) {
        for (int k = i; k < n - 1; k++) {
            *(ptrA + k) = *(ptrA + k + 1); // Décalage à gauche
        }
        n--; // Réduire la taille de A
        i--; // Revenir à l'indice précédent pour gérer le décalage
    }
}

// Affichage du tableau A après suppression
printf("Tableau A après suppression : ");
for (int i = 0; i < n; i++) {
    printf("%d ", *(ptrA + i));
}
printf("\n");

return 0;

```

Exercice 4 :

Ecrire une fonction qui permet de rechercher dans un tableau d'entiers T une valeur X.

```
void ChercherVal(int T[], int n, int X, int *pos, int *nbOcc);
```

Dans pos, on sauvegarde l'indice de la dernière apparition et -1 si la valeur n'a pas été trouvée. nbOcc indique le nombre d'occurrence de X dans T.

Solution

```
#include <stdio.h>

// Fonction pour rechercher une valeur X dans le tableau T
void ChercherVal(int T[], int n, int X, int *pos, int *nbOcc) {
    *pos = -1;    // Initialisation de l'indice de la dernière apparition à -1
    *nbOcc = 0;   // Initialisation du compteur d'occurrences à 0

    // Parcourir le tableau T
    for (int i = 0; i < n; i++) {
        if (T[i] == X) {
            *pos = i;        // Mise à jour de la position (dernière apparition)
            (*nbOcc)++;       // Incrémenter le compteur d'occurrences
        }
    }
}
```

Solution

```
int main() {  
  
    int i,T[50],n, X,pos, nbOcc;  
  
    // saisie de n  
    printf("Entrez la taille du tableau: ");  
    scanf("%d", &n);  
  
    // remplissage du tableau  
    for (i=0;i<n;i++)  
        scanf("%d", &T[i]);  
  
    // Saisie de la valeur à rechercher  
    printf("Entrez une valeur à rechercher : ");  
    scanf("%d", &X);  
  
    // Appel de la fonction ChercherVal  
    ChercherVal(T, n, X, &pos, &nbOcc);  
  
    // Affichage des résultats  
    if (pos != -1) {  
        printf("La valeur %d a été trouvée %d fois, dernière apparition à l'indice %d.\n", X, nbOcc, pos);  
    } else {  
        printf("La valeur %d n'a pas été trouvée dans le tableau.\n", X);  
    }  
  
    return 0;  
}
```